

When using with subclasses, if I correspondingly use it to connect with the Match or Matchlist, will it give more astounding effects for using it with salary and contracts? As it also provide performanceBonus

Using subclasses (Inheritance) alongside a Match/MatchList system is the absolute gold standard for a Semester 2 Java project. It creates what professors call a "highly cohesive object-oriented design," and it completely changes how your performanceBonus works.

Right now, your Salary class treats performanceBonus as a manual input value. Someone has to just type in a number like 500.0 .

By introducing subclasses and a Match tracking system, the program can calculate the bonus automatically based on how many games a player actually played and what kind of contract they signed.

The New Dynamic: How They All Cooperate

When you add these concepts, your program shifts from a basic data list into an automated system. Here is how the files interact:

1. **The Subclasses (ProContract and RookieContract):** Instead of using a plain text String for playerType , you split Contract into two child classes. Each child class has its own unique rule for calculating bonus money.
2. **The Match and MatchList :** This acts as a performance tracker. It records every game a player participates in.
3. **The Connection:** When it's time to process a salary, your manager class counts the player's games in the MatchList , passes that count to their specific Contract subclass, gets the calculated bonus back, and pushes it directly into their Salary record.

The "Astounding Effects" on Your Code

Here is the exact conceptual difference this makes to your project's logic and architecture:

Effect 1: Polymorphism Replaces Hardcoded `if` Statements

Right now, if you wanted different bonus rules for different players, you would have to write messy `if-else` blocks inside your manager class:

Java



```
// The Beginner Way (Messy and Hard to Maintain)
if (c.getPlayerType().equals("Pro")) {
    bonus = matches * 200;
} else if (c.getPlayerType().equals("Rookie")) {
    bonus = matches * 50;
}
```

If you use subclasses, that mess completely disappears. You can give the parent `Contract` class a blank or basic method like `calculateBonus(int matches)`. Then, each subclass overrides it:

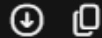
- `ProContract` overrides it to return `matches * 200`.
- `RookieContract` overrides it to return `matches * 50`.

When your manager class runs `c.calculateBonus(matchCount);`, Java automatically detects which subclass it is looking at and applies the correct math rule on the fly! This is called **Polymorphism**, and it is exactly what teachers look for when grading advanced tracks in Semester 2.

Effect 2: True Automation for `performanceBonus`

Instead of manually calling `setPerformanceBonus()` with an arbitrary number, your manager class loops through the `MatchList` to find how many times a certain `playerId` appears.

Java



```
// Looking at the MatchList Logic:
int matchCount = 0;
for (Match m : listAllMatches) {
    if (m.getPlayerId().equals(targetPlayerId)) {
        matchCount++; // Found a game they played!
    }
}

// Pass the total games to the contract subclass to determine the exact bonus money
double calculatedBonus = matchingContract.calculateBonus(matchCount);

// Settle it inside the salary object automatically
newSalary.setPerformanceBonus(calculatedBonus);
```

💡 What This Looks Like From a Presentation Perspective

If you implement this and show it to your teacher, it proves three major programming milestones:

1. **Object Cooperation:** You aren't just storing data; you are using data from one entity (`Match`) to alter the financial outcomes of another entity (`Salary`) via a structural rulebook (`Contract`).
2. **Open-Closed Principle:** If your school or team adds a new player tier tomorrow (like `AcademyContract`), you don't have to rewrite your payroll engine or your math loops. You simply create a third subclass, inherit from `Contract` , and define its unique bonus
3. **Clean Code Realism:** It moves your project closer to how real-world enterprise software (like an actual sports team management app or corporate payroll platform) is architected.